

Software Requirements Specification (SRS)

Endure and Survive

Team: Group 5

Authors: Ben Houle, Cam Gagne, Brandon Maldonado, Thomas McGeachie, Gia Panchal

Customers: Middle School Teachers and students in grades 4 through 6.

Instructor: Dr. James Daly

1 Introduction

This document provides an organized overview of Endure and Survive, an educational computer game designed to teach survival skills to students in grades 4–6. It begins with an introduction and a high-level description of the project before moving into detailed sections outlining the game’s requirements. These sections cover functional behavior, system modeling, and the abstract representations that describe how the game operates. A prototype is then presented to illustrate how these design elements connect in practice. The document concludes with the resources referenced throughout the project and the appropriate points of contact.

1.1 Purpose

The purpose of this document is to clearly define and communicate the design and functionality of Endure and Survive, an educational game created to teach foundational survival skills to young students. This Software Requirements Specification (SRS) provides the technical details, functional requirements, and supporting diagrams needed to guide developers from the initial concept to a working prototype.

In addition to supporting development, this document also presents the project to its broader audience—teachers, curriculum planners, and young students—by outlining the game’s educational goals and demonstrating how it can be used as an effective learning tool. The SRS serves as the central reference point for both the development team and stakeholders, ensuring that the game’s objectives and features are clearly understood throughout the creation process.

1.2 Scope

The software product to be developed is Endure and Survive, a 2D educational computer game created using the Godot engine. This application is intended for students in grades 4–6 and focuses on teaching essential survival skills through interactive, scenario-based gameplay. Its primary objectives are to promote critical thinking, problem-solving, and decision-making in real-world survival contexts while maintaining engagement through a visually simple yet immersive 2D environment. The application domain is educational gaming, specifically designed for classroom and home learning. The software will provide structured levels, intuitive controls, and age-appropriate challenges that reinforce learning outcomes. It will not include multiplayer functionality, advanced 3D graphics, or unrelated entertainment features. The game emphasizes accessibility, educational clarity, and ease of use to ensure both learning effectiveness and user enjoyment.

1.3 Definitions, acronyms, and abbreviations

- Endure and Survive: Name of the game
- SRS: Software Requirements Specifications
- HW: Hardware
- SW: Software
- Survival skills: Skills such as starting a fire, building a shelter, navigation, etc.
- Orienteering: The act of navigating using tools such as a map and compass
- Checkpoint: A point in a level where the player's progression is saved
- Badge: An in-game item that signifies the player's completion of a level
- Lean-to shelter: A simple, slanted shelter built by leaning sticks together.
- Tinder: Easily combustible material, such as dry leaves, used to start a fire.
- Kindling: Small sticks or twigs used to sustain a flame after the tinder ignites.

1.4 Organization

The remainder of the SRS document contains the following:

Section 2 provides the overall description of Endure and Survive and is divided into six subsections. Subsection 2.1 explains the game's context and describes how players interact with its various interfaces. Subsection 2.2 outlines the major system functions, including both the features the player directly engages with and the background processes that support gameplay. Subsection 2.3 defines the expected characteristics and behaviors of the player. Subsection 2.4 identifies potential constraints—such as software limitations, hardware considerations, or external requirements—that may influence development. Subsection 2.5 documents the assumptions made about the game's hardware, software, and users. Subsection 2.6 describes how specific requirements are distributed throughout the system.

Section 3 presents the detailed functional requirements for Endure and Survive.

Section 4 contains the modeling requirements, including all diagrams used to represent the system: Use Case, Class, Sequence, and State diagrams. Section 5 showcases a preview of the prototype, demonstrating what players will see, how the game is played, and several example scenarios. Section 6 lists the references used throughout this document, and Section 7 includes the points of contact for the project team.

2 Overall Description

This section provides an overall description of the product, organized into several parts. The Product Perspective explains the technical context of the game and how it fits within its environment. The Product Functions outline the core features of the game and summarize how players interact with it. The User Characteristics section describes the expected skills, behavior, and knowledge of the players or anyone teaching the game. The Constraints section identifies any technical or design limitations that may impact development. Assumptions and Dependencies describe the expected system, hardware, software, and user conditions required for the game to operate as intended. Finally, Apportioning of Requirements explains which aspects of the product are outside the scope of this development phase.

2.1 Product Perspective

Endure and Survive is a standalone 2D educational game designed to teach survival skills to students in grades 4–6. It is not part of a larger system but interacts with the user through a graphical interface and relies on standard hardware and operating systems. The game runs on desktop or laptop computers and uses the Godot engine for execution.

Interface Constraints

- System Interfaces: The game interacts with the operating system for file storage (progress data) and rendering graphics/audio.
- User Interfaces: A 2D graphical interface with keyboard and mouse input. Includes menus for level selection, checkpoint navigation, and feedback screens.
- Hardware Interfaces: Requires a keyboard, mouse, and an audio output device. Runs on standard desktop/laptop hardware.
- Software Interfaces: Compatible with Windows, macOS, and Linux.
- Communication Interfaces: Primarily offline; optional internet access for updates or leaderboard synchronization (if implemented).

Other Constraints

- Memory: Minimal storage for saving progress, badges, and leaderboard data (estimated <50 MB).
- Operations: Game must run smoothly on systems with basic specs (e.g., 4 GB RAM, integrated graphics).
- Site Adaptation Operations: No major customization required; installation involves downloading and running the executable.

2.2 Product Functions

Endure and Survive is a 2D educational game designed to teach survival skills through three structured levels. Each level tasks the player with exploring an island from a top-down perspective, and completing an objective relating to its given skill. The first level focuses on starting a fire, where players learn to identify and gather tinder, kindling, and larger fuel materials, construct a proper fire structure, and safely ignite the fire using provided tools. The second level introduces shelter building, requiring players to select a safe location, gather materials such as branches, leaves, and rope, and assemble a functional lean-to shelter. The third level teaches orienteering, guiding players to determine their current location using a map, understand cardinal directions, and navigate to assigned checkpoints using a map and compass.

The game uses a checkpoint-based progression system, where each survival task is represented as a checkpoint. Players must complete all checkpoints in a level before advancing to the next. Each checkpoint validates the correctness of the task and provides immediate feedback, visually indicating completed and remaining tasks. Progress is saved at every checkpoint, and players can replay completed checkpoints or retry failed levels without losing overall progress.

To ensure engagement and accessibility, the game presents instructions in short, clear sentences at a 4th–6th grade reading level, avoids technical terminology unless explained simply, and uses consistent visuals such as icons and images to support major tasks. Feedback is specific, identifying incorrect steps and offering corrective suggestions. The game also tracks progress, awards badges for level completion, and displays a leaderboard for the top three completion times. Replaying levels is encouraged through optional difficulty modifiers and randomized item locations. Failure conditions such as time limits or incorrect task execution are included, and players can retry levels from the start while retaining progress from previous levels.

2.3 User Characteristics

The intended users of Endure and Survive are students in grades 4–6, typically aged 9–12, with a basic level of computer literacy. They are expected to have prior experience using a keyboard and mouse for simple navigation and gameplay but do not require advanced technical skills. The game assumes no prior knowledge of survival strategies; instead, it introduces these concepts in an accessible, age-appropriate manner. Users should be comfortable following on-screen instructions, engaging with interactive elements, and applying critical thinking to solve challenges presented within the game environment.

2.4 Constraints

The game must be comprehensible and educational for students in grades 4–6, using language at an appropriate reading level and avoiding unexplained technical jargon. It must adhere to factual accuracy based on standard beginner survival guidelines to ensure safety and reliability. Safety-critical properties include preventing the presentation of incorrect or misleading survival instructions, as this could result in harmful misconceptions. The game must also avoid exposing students to inappropriate content or unsafe practices, ensuring compliance with educational standards and age-appropriate ratings. Other essential properties include maintaining functional checkpoint validation, consistent feedback mechanisms, and proper progress tracking. If these properties are violated, such as by failing to save progress, providing inaccurate feedback, or allowing advancement without completing required tasks, the system will not perform its intended educational function effectively. Additionally, the game must remain stable across supported platforms and provide clear failure conditions to prevent confusion during gameplay.

2.5 Assumptions and Dependencies

The game is assumed to run on standard desktop or laptop computers equipped with a keyboard, mouse, and an audio output device, using a modern operating system such as Windows, macOS, or Linux. A stable internet connection may be required for downloading updates or accessing online resources, but core gameplay is designed to function offline. The software environment includes a Godot-based executable or project build, ensuring compatibility with systems that meet Godot's minimum requirements. Users are expected to interact primarily through keyboard inputs for movement or actions, and occasional drag-and-drop or point-and-click mechanics. The physical environment should be a classroom or home setting with minimal distractions, allowing students to focus on the educational content.

2.6 Apportioning of Requirements

For the prototype of Endure and Survive, three levels are planned, based on three survival skills. Later in development, more levels may be added to expand the scope of the game and incorporate more relevant skills and challenges. If this is the case, any new levels will be incorporated in a modular way that allows for the content of the game to expand without necessitating significant changes to the engine. In addition, they will be created in the same style as the original three, to create a cohesive and consistent feel to the game as a whole.

3 Specific Requirements

1. The game shall teach players survival skills through multiple levels.

1.1. The game shall include a minimum of three survival skill learning levels.

1.1.1. *Level 1- Starting a fire:*

1.1.1.1. The player shall identify and gather tinder and kindling.

1.1.1.2. The player shall identify and gather larger fuel materials.

1.1.1.3. The player shall construct a proper fire structure.

1.1.1.4. The player shall safely ignite a fire using provided tools.

1.1.2. *Level 2- Basic shelter building (Lean-to):*

1.1.2.1. The player shall identify a safe shelter location.

1.1.2.2. The player shall gather the necessary materials such as branches, leaves, and rope.

1.1.2.3. The player shall assemble a functional lean-to shelter.

1.1.3. *Level 3- Orienteering (map + compass):*

1.1.3.1. The player shall determine their current location using a map.

1.1.3.2. The player shall demonstrate understanding of cardinal directions and orientation.

1.1.3.3. The player shall navigate to assigned checkpoints using map and compass.

2. The game shall have a checkpoint-based level progression system.

2.1. The game shall prompt the player to advance to the next level only after all required checkpoints for the current level are completed.

2.2. Each checkpoint shall represent a discrete survival task (e.g., gathering materials, building a fire, completing navigation to a location).

2.3. Checkpoints shall provide immediate feedback when completed correctly or incorrectly.

2.4. The game shall visually indicate which checkpoints have been completed and which remain.

2.5. The player shall be able to select any completed checkpoint when beginning or replaying a level.

3. The game shall be engaging and educational for young students.

3.1. The game shall use text written at approximately a 4th–6th grade reading level, avoiding complex vocabulary and long sentences.

3.1.1. The game shall avoid technical jargon unless it is explained in one sentence or less.

3.1.2. The game shall present key survival concepts using short paragraphs (no more than 2–3 sentences each).

- 3.2. The game shall provide step-by-step instructions at the start of each level, with each step shown as a separate bullet or visual cue.
- 3.3. The game shall present survival information that is factually accurate, consistent with standard beginner camping/survival guidelines (e.g., proper fire-building steps, safe shelter placement).
- 3.4. The game shall provide specific feedback, such as:
 - identifying the exact step the player did incorrectly,
 - offering a corrective suggestion, or
 - confirming successful task completion.
- 3.4.1. The game shall provide on-screen feedback immediately after passing a checkpoint or meeting a failure condition.
- 4. **The game shall include a system for tracing player progress.**
 - 4.1. The game shall track completed checkpoints and time spent.
 - 4.2. The game shall reward badges upon completion of all checkpoints of a level.
 - 4.3. The game shall list the top three players with the best completion times.
- 5. **The game shall encourage replayability.**
 - 5.1. The game shall randomize item locations each time the level is replayed.
- 6. **The game shall include clear failure conditions.**
 - 6.1. Each level shall include at least one condition that results in failure (e.g., running out of time, incorrect structure, or navigating off-path).
- 7. **The game shall allow players to retry levels.**
 - 7.1. Upon failing a level, the player shall have the option to retry.
 - 7.1.1. Retrying a level shall reset all progress in the level, including all checkpoints, collected items, and time spent for that level.
 - 7.1.2. The player shall start the level from the initial starting point.
 - 7.1.3. Progress from previously completed levels (checkpoints, badges, etc.) shall remain intact.

4 Modeling Requirements

4.1 Use Case Diagram

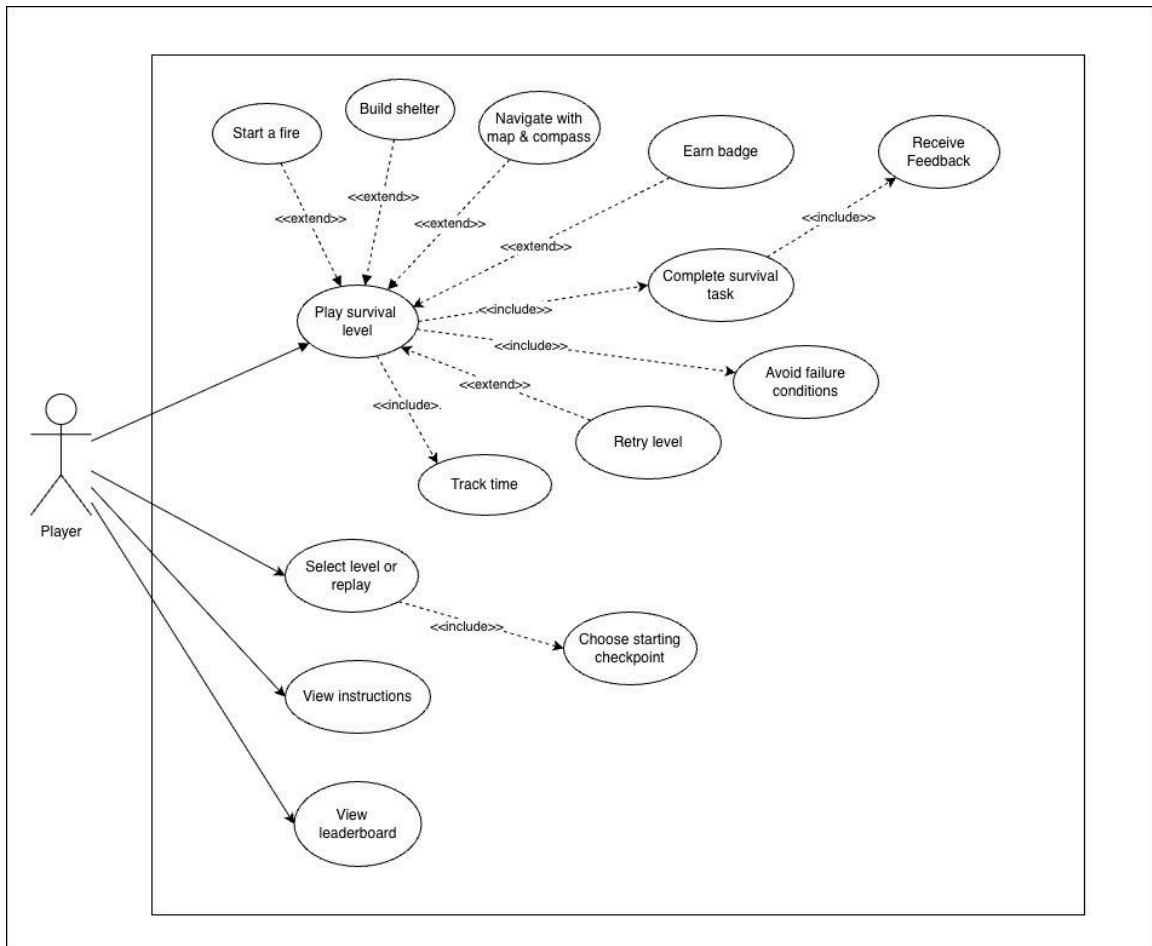


Figure 1: Use Case Diagram

The use case diagram shows how the player interacts with the Survive the Island game. Each oval represents a player goal or system-supported action. Solid lines indicate direct actions initiated by the player, such as starting gameplay, selecting a level, or viewing instructions. Dashed lines represent relationships between use cases: «include» for steps that are always part of another action (for example, completing survival tasks and receiving feedback during gameplay) and «extend» for optional or conditional behaviors (such as retrying a level after failure or earning a badge upon success). This diagram highlights the main flow—playing a survival level—along with supporting actions like selecting levels and checkpoints, avoiding failure conditions, and viewing progress through badges and leaderboards.

Use Case Name:	Play Survival Level
Actors:	Player
Description:	The player engages in a survival level to complete tasks and learn skills. Each time the player starts a level the item locations will be randomized.
Type:	Primary
Includes:	Complete survival task, Avoid failure conditions, Track time
Extends:	Build fire, Build shelter, Navigate with map & compass, Earn badges
Cross-refs:	Requirement 1.1.1.2 - 1.1.1.4, 1.1.2.1 - 1.1.2.3, 1.1.3.1 - 1.1.3.3, 5.1
Uses cases:	None

Use Case Name:	Select Level or Replay
Actors:	Player
Description:	The player chooses which level to play or replay. Each time the player starts or replays a level the item locations will be randomized.
Type:	Primary
Includes:	Choose starting checkpoint
Extends:	None
Cross-refs:	Requirement 1.1, 5.1, 7.1
Uses cases:	None

Use Case Name:	View instructions
Actors:	Player
Description:	The player views step-by-step instructions before or during play.
Type:	Primary
Includes:	None
Extends:	None
Cross-refs:	Requirement 3.1, 3.2, 3.3
Uses cases:	None

Use Case Name:	View leaderboard
Actors:	Player
Description:	The leaderboard will show a list of the top three players with the best completion times.
Type:	Primary
Includes:	None
Extends:	None
Cross-refs:	Requirement 4.3
Uses cases:	None

Use Case Name:	Start a fire
Actors:	Player
Description:	The first level is starting a fire, learning how to gather tinder and kindling.
Type:	Secondary
Includes:	None
Extends:	None
Cross-refs:	Requirement 1.1.1
Uses cases:	None

Use Case Name:	Build shelter
Actors:	Player
Description:	The second level is building a basic shelter, collecting material and creating a basic structure.
Type:	Secondary
Includes:	None
Extends:	None
Cross-refs:	Requirement 1.1.2
Uses cases:	None

Use Case Name:	Navigating with map & compass
Actors:	Player
Description:	The final level is orienteering, learning how to use a map and compass.
Type:	Secondary
Includes:	Select checkpoint
Extends:	None
Cross-refs:	Requirement 1.1.3
Uses cases:	None

Use Case Name:	Track time
Actors:	Player
Description:	The player keeps track of time using the timer in the game.
Type:	Secondary
Includes:	None
Extends:	None
Cross-refs:	Requirement 4.1, 4.3
Uses cases:	None

Use Case Name:	Earn badge
Actors:	Player
Description:	This will provide a visual for completion of level and will reward the user with a badge that will stay even if they choose to retry the level.
Type:	Secondary
Includes:	None
Extends:	None
Cross-refs:	Requirement 4.2
Uses cases:	None

Use Case Name:	Retry level
Actors:	Player
Description:	A user has the ability to retry a level if any of the failure conditions are met. Retrying a level will reset all level-specific progress, including all checkpoints, collected items, and time spent for that level. The player will start the level from the initial starting point.
Type:	Secondary
Includes:	None
Extends:	None
Cross-refs:	Requirement 7.1 - 7.1.1, 7.1.2, 7.1.3
Uses cases:	None

Use Case Name:	Complete survival task
Actors:	Player
Description:	Completing survival tasks includes completing all checkpoints and the user can now move on to the next level.
Type:	Secondary
Includes:	Receive feedback
Extends:	None
Cross-refs:	Requirement 2.1
Uses cases:	None

Use Case Name:	Receive Feedback
Actors:	Player
Description:	Feedback is provided in clear language each time the user completes a checkpoint and/or meets a failure condition.
Type:	Secondary
Includes:	None
Extends:	None
Cross-refs:	Requirement 2.3, 3.1, 3.3, 3.4 - 3.4.1
Uses cases:	None

Use Case Name:	Avoid failure conditions
Actors:	Player
Description:	Each level will have at least one condition that results in failure (e.g., running out of time, incorrect structure, or navigating off-path). It is the player's role to avoid these and achieve success.
Type:	Secondary
Includes:	None
Extends:	None
Cross-refs:	Requirement 6.1
Uses cases:	None

Use Case Name:	Choose starting checkpoint
Actors:	Player
Description:	Once a user chooses the level they would like to play, they are next prompted with the checkpoints they would like to play. The choices for the checkpoints will include ones they have already completed and the most recent unplayed.
Type:	Secondary
Includes:	None
Extends:	None
Cross-refs:	Requirement 2.1, 2.2, 2.4, 2.5
Uses cases:	None

4.2 Class Diagram

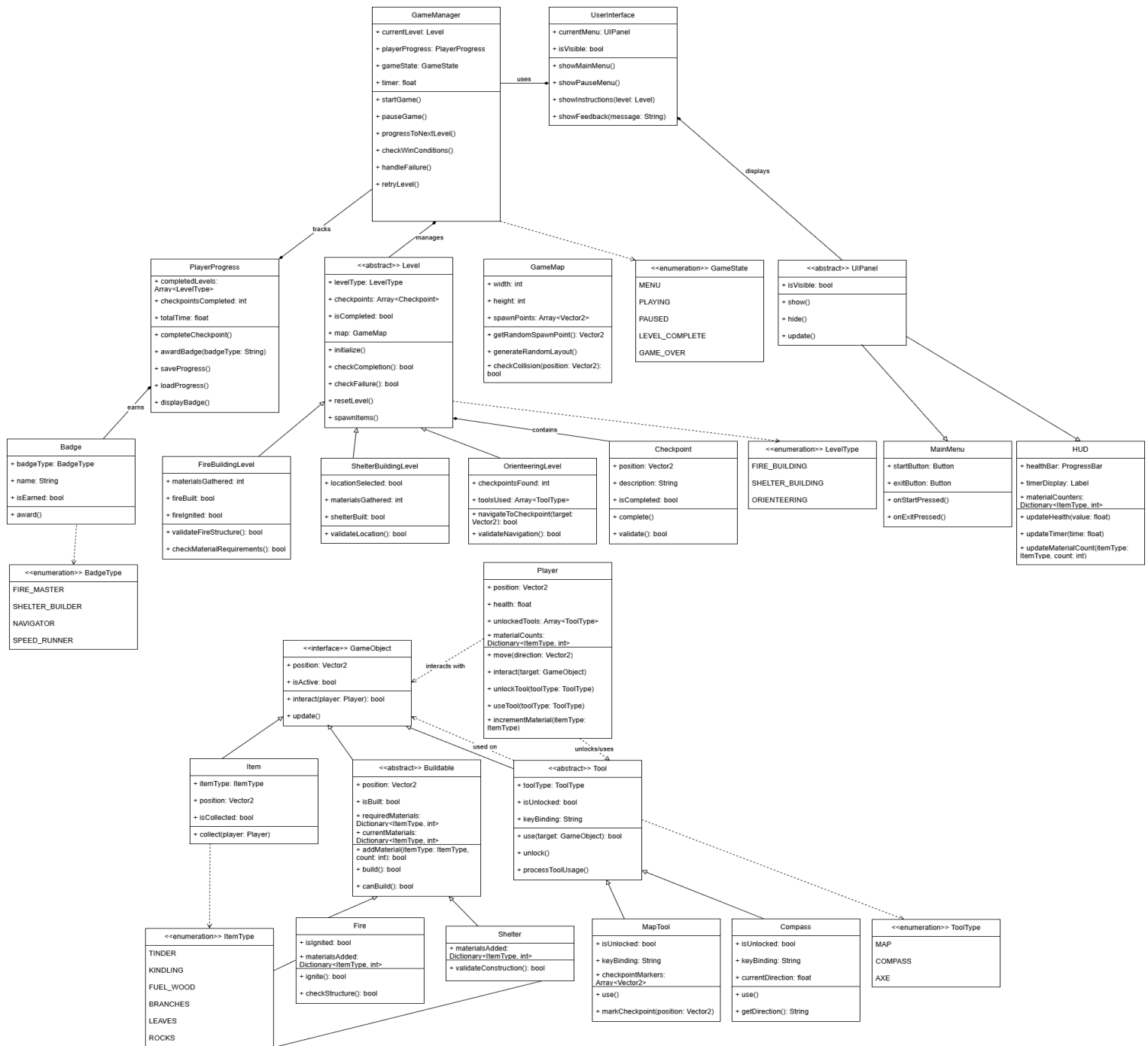


Figure 2: Class Diagram

This class diagram illustrates how the main components of *Endure and Survive* work together during gameplay. It shows the Player, Level, Environment, and Item classes, along with how they interact—for example, the Player depends on the Level for objectives, and Levels contain the objects and hazards shown in the game. The attributes and methods in each class reflect what they manage in the game world, while the relationships between them show how data and actions flow during play.

Data Dictionary

Element Name		Description
Badge		Represents an achievement or award the player can earn by completing specific challenges.
Attributes		
	name: String	Badge name
	isEarned: bool	Whether the player has earned a badge or not.
Operations		
	award()	Gives a player the earned badge.
Relationships	PlayerProgress -> Badge (Composition): PlayerProgress contains and manages all Badge instances earned by the player. Badge -> BadgeType (Dependency): Badge depends on the BadgeType enumeration to categorize badges.	
UML Extensions	The Badge class does not use stereotypes, tags, or constraints.	

Element Name		Description
BadgeType		An enumeration representing all possible badge categories that a player can earn in the game.
Attributes		
	FIRE_MASTER	Earned for successfully completing fire-building level.
	SHELTER_BUILDER	Earned for successfully completing shelter-building level.

	NAVIGATOR	Earned for successfully completing orienteering level.
Operations		
	There are no operations	
Relationships	Badge -> BadgeType (Dependency): The Badge class depends on this enumeration because each Badge includes a BadgeType attribute whose value comes from this enum.	
UML Extensions	This class uses the <<enumeration>> stereotype.	

Element Name		Description
Buildable		An abstract base class representing any structure that the player can build in game.
Attributes		
	position: Vector2	The location of the buildable structure.
	isBuilt: bool	Whether the buildable structure has been successfully completed.
	requiredMaterials: Dictionary<ItemType, int>	A list of materials needed to construct the structure.
	currentMaterials: Dictionary<ItemType, int>	A list of materials the player currently has.
Operations		
	addMaterial(itemType, count): bool	Adds materials to the buildable. Returns true if accepted.
	build(): bool	Completes the building using current materials. Returns true if successful.
	canBuild(): bool	Checks whether the required materials have been fully added.
Relationships	Buildable is the superclass of Fire and Shelter. GameObject -> Buildable: Buildable implements the GameObject interface. Buildable -> ItemType: Buildable depends on ItemType because its requiredMaterials and currentMaterials use ItemType keys.	
UML Extensions	Uses <<abstract>>.	

Element Name		Description
Checkpoint		A checkpoint used to track player progress.
Attributes		
	position: Vector2	The checkpoint's coordinates within the level
	description: String	A short explanation for this checkpoint
	isCompleted: bool	Tracks whether the checkpoint has already been completed by the player.
Operations		
	complete()	Marks the checkpoint as completed.
	validate(): bool	Checks whether conditions to complete the checkpoint have been met.
Relationships	Level -> Checkpoint: A checkpoint cannot exist without its level.	
UML Extensions	The Checkpoint class does not use stereotypes, tags, or constraints.	

Element Name		Description
Compass		A tool that provides directional information to the player.
Attributes		
	isUnlocked(): bool	Indicates whether the player has unlocked the compass.
	keyBinding(): String	The keyboard key is assigned to activate the compass.
	currentDirection: float	Represents the compass's current facing direction
Operations		
	use(): void	Activates the compass to provide directional guidance to the player.
	getDirection(): String	Returns a representation of the direction the compass is pointing.
Relationships	Tool -> Compass: Compass is a subclass of the abstract Tool class. The player uses Compass after unlocking.	

UML Extensions	The Compass class does not use stereotypes, tags, or constraints.
----------------	---

Element Name		Description
Fire		A buildable structure representing a campfire in the game
Attributes		
	isIgnited: bool	Indicates whether the fire is currently burning.
	materialsAdded: Dictionary<ItemType, int>	Tracks materials the player has added toward building and igniting the fire.
Operations		
	ignite(): bool	Attempts to ignite the fire. Returns true if successful.
	checkStructure(): bool	Validates whether the fire has been constructed with the correct arrangement of materials.
Relationships	Fire is a concrete subclass of Buildable. Fire implements GameObject through Buildable. FireBuildingLevel uses the Fire object to determine whether required conditions are met.	
UML Extensions	The Fire class does not use stereotypes, tags, or constraints.	

Element Name		Description
FireBuildingLevel		A concrete subclass of Level centered around collecting materials and constructing/igniting a fire.
Attributes		
	materialsGathered: int	Number of fire-building materials the player has collected.
	fireBuilt: bool	Indicates whether the basic structure of the fire has been assembled.
	fireIgnited: bool	Tracks whether the completed fire has been successfully ignited.
Operations		

	validateFireStructure(): bool	Checks whether the fire structure meets the necessary requirements to be ignited.
	checkMaterialRequirements() : bool	Checks whether the player has gathered the required materials for building the fire.
Relationships	FireBuildingLevel is a concrete Level type.	
UML Extensions	The FireBuildingLevel class does not use stereotypes, tags, or constraints.	

Element Name		Description
GameManager		Central controller that manages the overall game flow, level progression, and win/fail logic.
Attributes		
	currentLevel: Level	The current level being played.
	playerProgress: PlayerProgress	Tracks the player's long term progress, including completed levels and earned badges.
	gameState: GameState	The current state of the game (running, paused).
	timer: float	Tracks elapsed time for the current level.
Operations		
	startGame(): void	Initializes game systems and loads a level.
	pauseGame(): void	Pauses gameplay and updates UI.
	progressToNextLevel(): void	Ends current level and loads the next one.
	checkWinConditions(): bool	Determines whether the player has met the requirements to win.
	handleFailure(): void	Executes failure logic, UI updates, and resetting.
	retryLevel(): void	Restarts the current level from the beginning.
Relationships	Uses GameMap to load and manage level content. Controls GameState transitions. Communicates with HUD to update UI during gameplay. Manages level objects. Updates PlayerProgress.	

UML Extensions	The GameManager class does not use stereotypes, tags, or constraints.
----------------	---

Element Name		Description
GameMap		Represents the in-game environment layout, including terrain, interactable objects within a level.
Attributes		
	width: int	Width of the game map.
	height: int	Height of the game map.
	spawnPoints: Array<Vector2>	An array of coordinates for the spawn points.
Operations		
	getRandomSpawnPoints(): Vector2	Chooses a random spawn point from the array.
	generateRandomLayout(): void	Creates a unique map layout.
	checkCollision(position: Vector2): bool	Checks for collision between player's position and map objects.
Relationships	Level uses GameMap to create the level layout.	
UML Extensions	The GameMap class does not use stereotypes, tags, or constraints.	

Element Name		Description
GameObject		Interface defining all interactive in game objects.
Attributes		
	position: Vector2	Object's world position
	isActive: bool	Whether the object is active in the game.
Operations		
	interact(player: Player): bool	Handles interaction logic with the player.
	update(): void	Updates the object behavior each frame.
Relationships	The player interacts with GameObject. Tool used on GameObject.	
UML Extensions	Uses <<interface>> stereotype.	

Element Name		Description
GameState		Enumeration defining all possible game states.
Attributes		
	MENU	
	PLAYING	
	PAUSED	
	LEVEL_COMPLETE	
	GAME_OVER	
Operations		
Relationships	Depended on by GameManager	
UML Extensions	Uses <<enumeration>> stereotype.	

Element Name		Description
HUD		Heads Up Display showing real-time player information such as time and materials.
Attributes		
	healthBar: ProgressBar	Shows a player's current health.
	timerDisplay: Label	Displays the active game timer.
	materialCounters: Dictionary<ItemType, int>	Shows collected materials and counts.
Operations		
	updateHealth(value: float)	Updates the health bar.
	updatesTimer(time: float)	Updates the displayed timer.
	updateMaterialCount(itemType: ItemType, count: int)	Updates the materials count display.
Relationships	Subclass of UIPanel.	
UML Extensions	The HUD class does not use stereotypes, tags, or constraints.	

Element Name	Description
--------------	-------------

Item		An in game collectible object that the player can pick up.
Attributes		
	itemType: ItemType	Identifies what type of item it is.
	Position: Vector2	The item's location in the world.
	isCollected: bool	Whether the item has been picked up by the player.
Operations		
	collect(player: Player)	Allows a player to pick up the item and update material counts.
	OperationName2 (arglist): returnType1	Description4
Relationships	Implements GameObject. Uses ItemType. Interacted with by Player.	
UML Extensions	Implements <<interface>> GameObject.	

Element Name		Description
ItemType		Enumeration representing all possible collectible materials types in the game.
Attributes		
Operations		
Relationships	Used by Item. Used in Player's materialCounts dictionary. Used in Buildable.	
UML Extensions	Uses UML stereotype <<enumeration>>	

Element Name		Description
Level		Abstract base class for all level types. Handles level setup, completion checks, failure conditions, and item spawning
Attributes		
	levelType: LevelType	Identifies which type of level this is.
	checkpoints: Array<Checkpoint>	List of checkpoints required for completion.

	isCompleted: bool	Whether the level has been completed.
	map: GameMap	The map used by this level.
Operations		
	initialize()	Set up the level when it begins.
	checkCompletion(): bool	Determines whether the level is finished.
	checkFailure(): bool	Determines if the player has failed the level.
	resetLevel()	Restores the level to its initial state.
	spawnItems()	Generates items on the map.
Relationships	Level is the superclass of FireBuilderLevel, ShelterBuilderLevel, and OrienteeringLevel. Composes Checkpoint and GameMap. Depends on LevelType.	
UML Extensions	Declared as <<abstract>>.	

Element Name		Description
LevelType		Enumeration specifying the different categories of levels in the game.
Attributes		
Operations		
Relationships	Used by Levels.	
UML Extensions	Uses stereotype <<enumeration>>.	

Element Name		Description
MainMenu		A specific UI panel representing the game's main menu, containing start and exit controls.
Attributes		
	startButton: Button	UI button used to begin the game.
	exitButton: Button	UI button used to quit the game.
Operations		
	onStartPressed()	Called when the user presses the Start button.
	onExitPressed()	Called when the user presses the Exit button.

Relationships	Subclass of UIPanel. Displayed by UserInterface.
UML Extensions	The MainMenu class does not use stereotypes, tags, or constraints.

Element Name		Description
MapTool		A tool that allows the player to view the map during gameplay.
Attributes		
	isUnlocked: bool	Whether the player has unlocked the map tool.
	keyBinding: String	The keyboard button is assigned to activate the tool.
	checkpointMarkers: Array<Vector2>	A list of checkpoint marker positions displayed on the map.
Operations		
	use(): void	Activates the map tool.
	markCheckpoint(position: Vector2): void	Place a marker at the specified position.
Relationships	Inherits from Tool. Implements GameObject interface. Used by Players.	
UML Extensions	The MapTool class does not use stereotypes, tags, or constraints.	

Element Name		Description
OrienteeringLevel		A level that focuses on navigation and finding checkpoints using map and compass tools.
Attributes		
	checkpointsFound: int	Number of checkpoints the player has discovered.
	toolsUsed: Array<ToolType>	List of tools the player used during the level.
Operations		
	navigateToCheckpoint(target: Vector2): bool	Attempts to navigate to a target checkpoint location.
	validateNavigation(): bool	Checks if the player's navigation is correct.

Relationships	Subclass of Level. Contains Checkpoints through Level. Uses GameMap.
UML Extensions	The OrienteeringLevel class does not use stereotypes, tags, or constraints.

Element Name		Description
Player		Represents the main character controlled by the user, including movement, collected materials, and tool interaction.
Attributes		
	position: Vector2	Current position of the player.
	health: float	Health value of the player.
	unlockedTools: Array<ToolType>	List of tools the player has unlocked.
	materialCounts: Dictionary<ItemType, int>	Materials collected by the player, by item type.
Operations		
	move(direction: Vector2): void	Moves the player in a given direction.
	interact(target: GameObject): void	Interacts with objects in the world.
	unlockTool(toolType: ToolType): void	Unlock a tool so it can be used.
	useTool(toolType: ToolType): void	Uses a specific tool.
	incrementMaterial(itemType: ItemType): void	Adds one or more materials to the player's inventory.
Relationships	Associates with GameObjects by interacting with them.	
UML Extensions	The Player class does not use stereotypes, tags, or constraints.	

Element Name		Description
PlayerProgress		Tracks the player's long term progress, achievements, and badge rewards.
Attributes		

	completedLevels: Array<LevelType>	Levels the player has finished.
	checkpointsCompleted: int	Total number of checkpoints completed across all levels.
	totalTime: float	Total gameplay time.
Operations		
	completeCheckpoint(): void	Marks a checkpoint as completed.
	awardBadge(badgeType: String): void	Gives the player a badge.
	saveProgress(): void	Saves progress to storage.
	loadProgress(): void	Loads progress from storage.
	displayBadge(): void	Shows a badge to the user.
Relationships	Aggregated by GameManager. Composes Badge.	
UML Extensions	The PlayerProgress class does not use stereotypes, tags, or constraints.	

Element Name		Description
Shelter		A buildable object used to complete shelter building objectives.
Attributes		
	materialsAdded: Dictionary<ItemType, int>	Materials the player has placed into the shelter construction.
Operations		
	validateConstruction(): bool	Check whether the shelter is built correctly.
Relationships	Subclass of Buildable. Implements GameObject interface	
UML Extensions	The Shelter class does not use stereotypes, tags, or constraints.	

Element Name		Description
ShelterBuildingLevel		A level focused on building a shelter by gathering materials and selecting an appropriate construction location.
Attributes		
	locationSelected: bool	Whether the player has chosen a valid shelter location.

	materialsGathered: int	Number of required materials collected.
	shelterBuilt: bool	Whether the shelter is fully built.
Operations		
	validateLocation(): bool	Checks if the selected shelter location is valid for construction.
Relationships	Subclass of Level. Contains Checkpoints via Level composition. Uses GameMap via Level.	
UML Extensions	The ShelterBuildingLevel class does not use stereotypes, tags, or constraints.	

Element Name		Description
Tool		An abstract tool class representing any usable equipment the player can activate.
Attributes		
	toolType: ToolType	Enumeration describing which tool this is.
	isUnlocked: bool	Whether the player has unlocked this tool.
	keyBinding: String	Assigned key/button to activate the tool.
Operations		
	use(target: GameObject): bool	Uses the tool on a specified game object.
	unlock(): void	Unlock the tool so the player can use it.
	processToolUsage(): void	Processes tool specific behavior after activation
Relationships	Implements GameObject. Superclass of MapTool and Compass. Used by Players.	
UML Extensions	Is an <<abstract>> stereotype.	

Element Name		Description
ToolType		Enumeration defining all tool types available to the player.

Attributes		
	MAP	
	COMPASS	
	AXE	
Operations		
Relationships	Used by Tool and Player.	
UML Extensions	Uses <<enumeration>> stereotype.	

Element Name		Description
UIPanel		Abstract base class for all UI panels.
Attributes		
	isVisible: bool	Whether the UI panel is shown on screen.
Operations		
	show(): void	Displays the UI panel.
	hide(): void	Hides the UI panel.
	update(): void	Updates the UI elements displayed within the panel.
Relationships	Superclass of MainMenu and HUD. Displayed by UserInterface.	
UML Extensions	<<abstraction>> stereotype.	

Element Name		Description
UserInterface		Manages all UI panels, menus, notifications, and feedback shown to the player.
Attributes		
	currentMenu: UIPanel	The UI panel is currently visible to the player.
	isVisible: bool	Whether the user interface is currently shown.
Operations		
	showMainMenu(): void	Displays the main menu.
	showPauseMenu(): void	Displays the pause menu.

	showInstructions(level: Level): void	Displays the instructions for the selected level.
	showFeedback(message: String): void	Shows on screen notifications or feedback.
Relationships	Aggregates UIPanel. Used by GameManager. Uses Level through showInstructions(level).	
UML Extensions	The UserInterface class does not use stereotypes, tags, or constraints.	

4.3 Sequence Diagrams

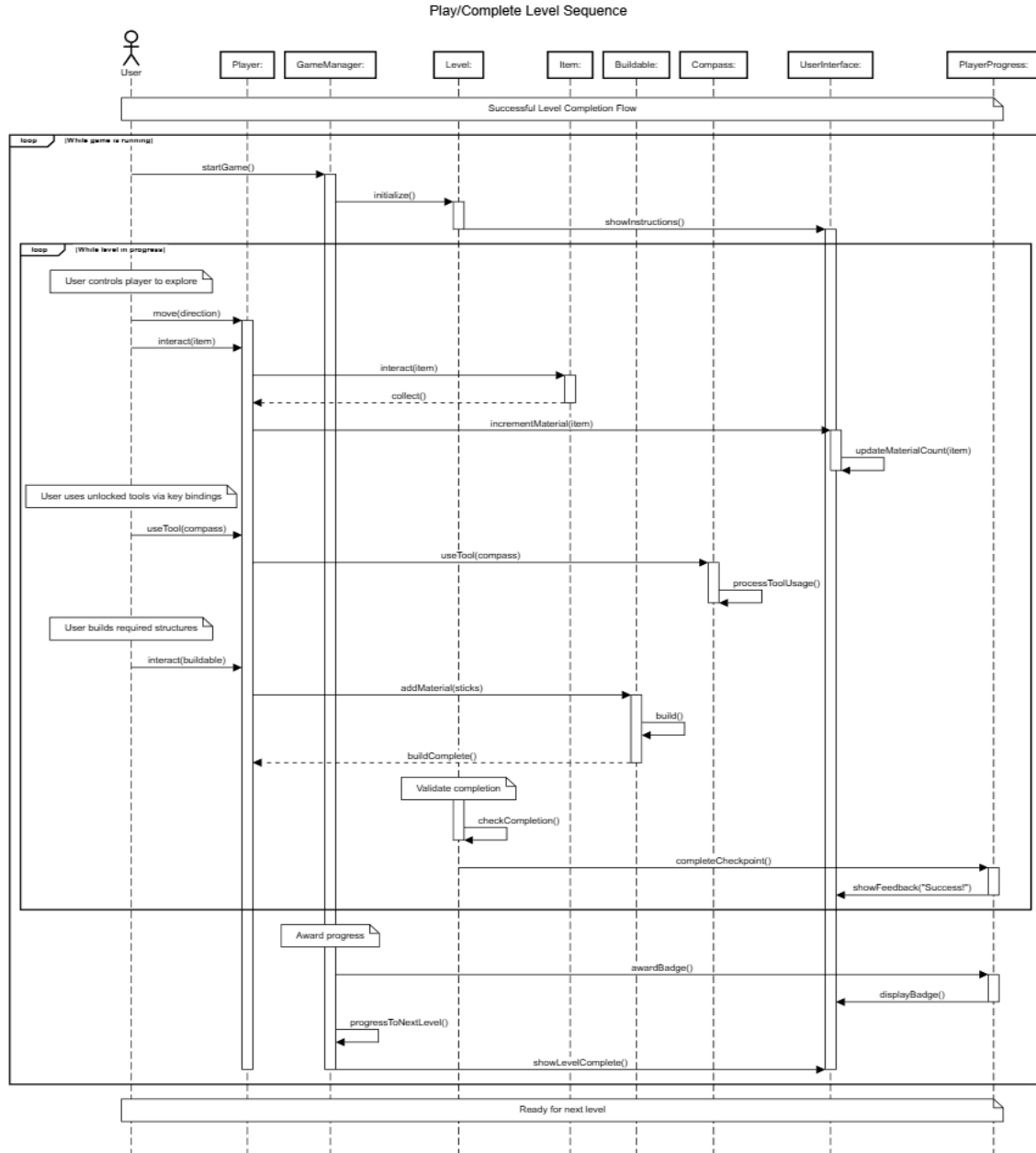


Figure 3: Play/Complete Level Sequence Diagram

The first sequence diagram illustrates the steps for successfully completing a level. The player explores, collects materials, and uses tools to build required structures. Each action updates progress and validates checkpoints. When all tasks are completed, the game confirms success, shows feedback, awards badges, and updates progress tracking.

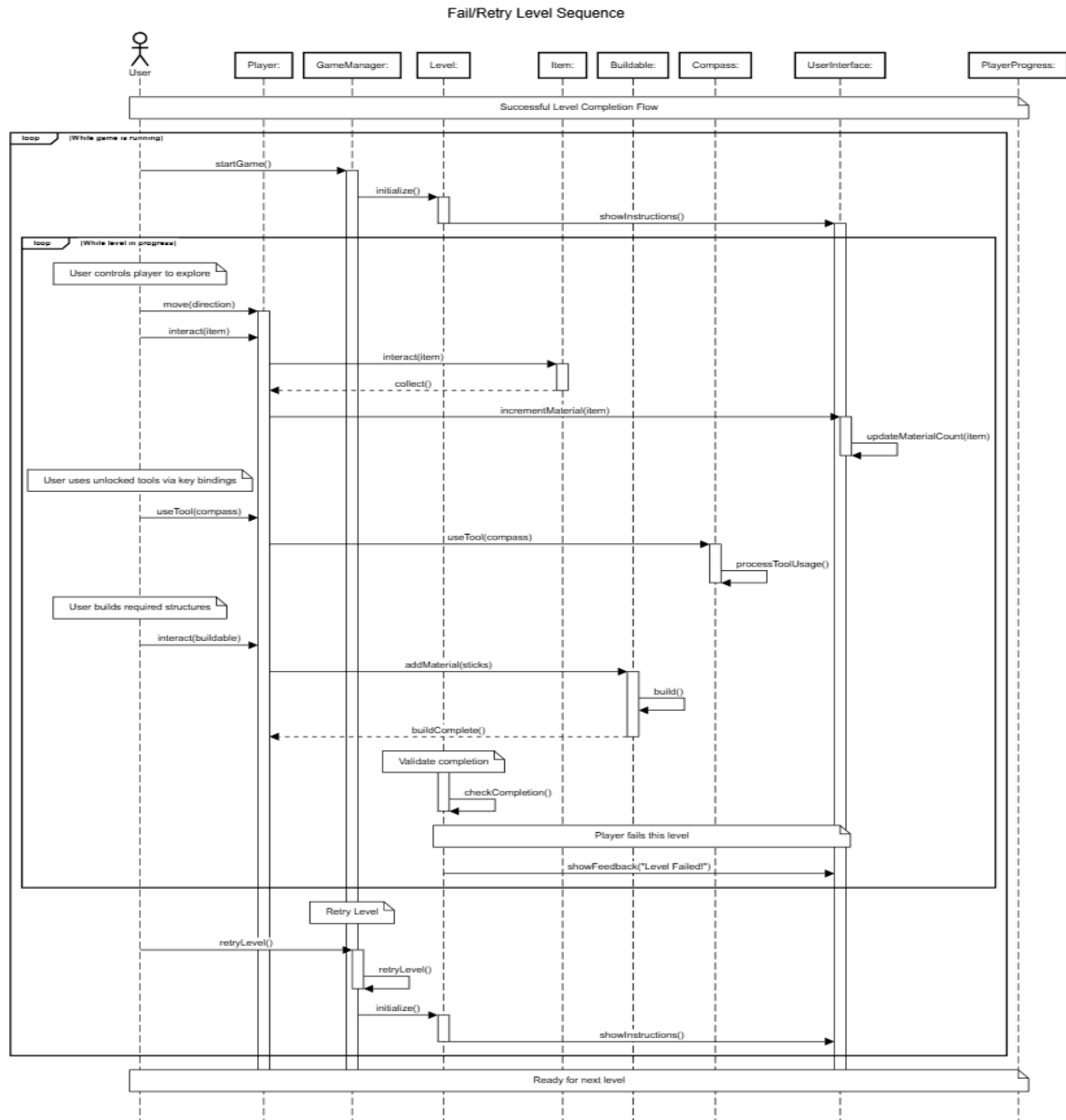


Figure 4: Fail/Retry Level Sequence Diagram

The second sequence diagram shows what happens when a player fails a level and chooses to retry. It starts with the player interacting with the game to explore, gather items, and use tools. The system checks progress at each checkpoint and validates tasks. If the player fails, the game displays feedback and offers the option to retry. When retry is selected, the game manager starts the level again, and the timer restarts so the player can attempt the tasks again.

4.4 State Diagram

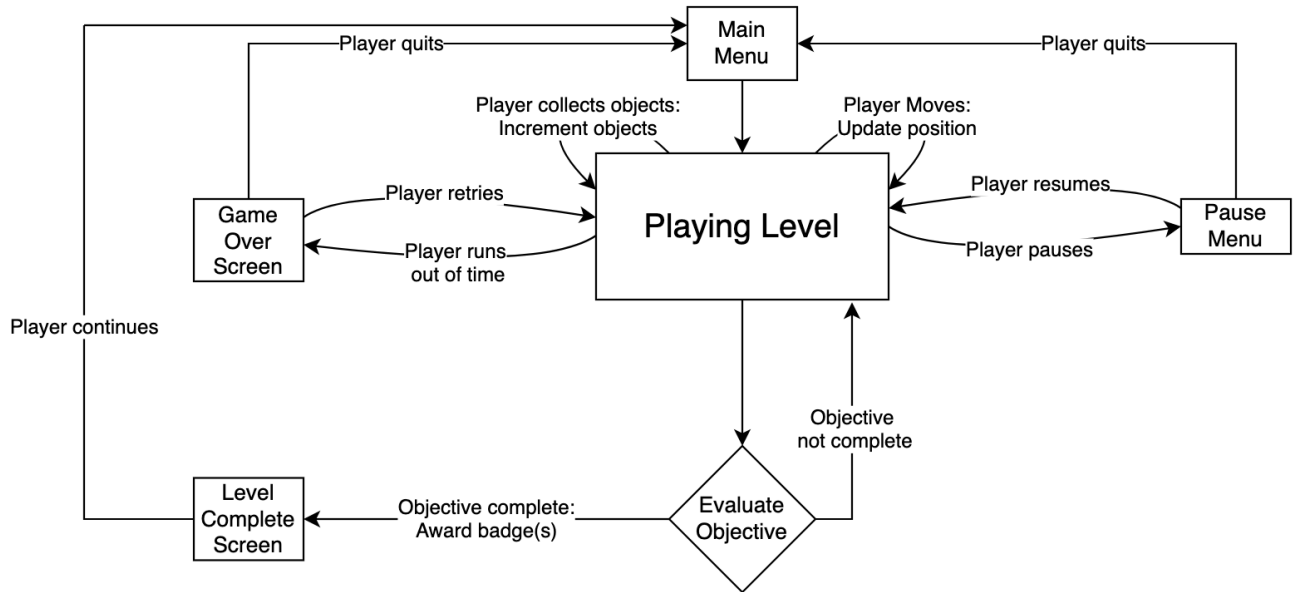


Figure 5: State Diagram

This state diagram represents the flow of a game with multiple screens and conditions. The central state is Playing Level, where the player moves and collects objects. From here, the player can pause the game, leading to the Pause Menu, or quit to the Main Menu. If the player runs out of time, the game transitions to the Game Over Screen, where they can retry or return to the main menu. When objectives are evaluated, if they are complete, the game moves to the Level Complete Screen, awarding badges and allowing the player to continue. If objectives are not complete, the game remains in the playing state. This diagram clearly shows how user actions and game conditions drive transitions between menus and gameplay states.

5 Prototype

The prototype for Endure and Survive is intended to show off the basic survival mechanics core to the game, as well as the overall structure. The prototype allows the player to select three levels, the second two being unlocked after the first is completed, each relating to a different survival skill. Each level also features a time limit which the player must complete the objective under.

The first level tasks the player with collecting various types of wood to create a small fire. There are three types of wood: sticks, kindling, and fuel, which are randomly placed throughout the level, and mixed among wet wood that isn't usable. The player must collect enough of each type of fuel before finding a suitable location to build a fire, at which point they are prompted to create a fire by interacting with a drag and drop menu.

The second level involves creating a lean-to shelter by collecting a number of logs, also randomized throughout the level. Once five logs are collected, the player must find a clearing to build a shelter using a similar drag and drop menu.

The final implemented level involves navigating the island using a provided compass, which points the player to two specific points on the map by displaying a cardinal direction. Once the player has navigated to both points, they successfully complete the level.

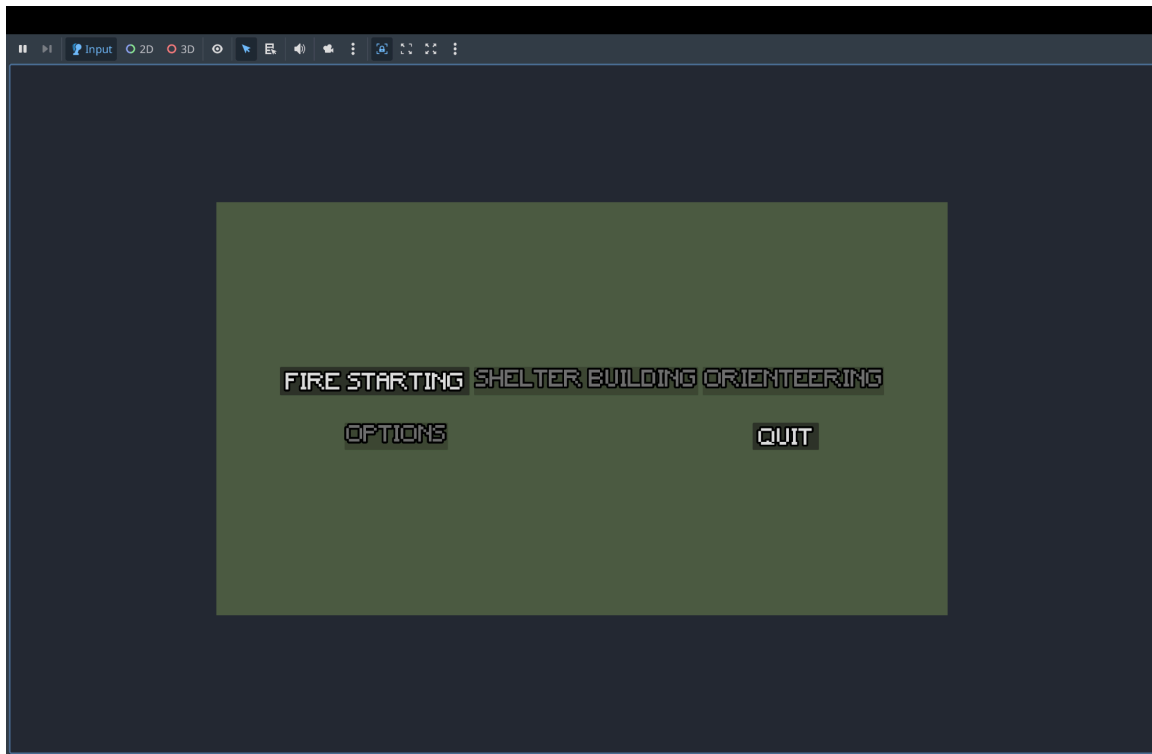
In addition, the prototype features a fully functioning UI which provides both a main menu to select a level, and a pause menu during a level, allowing the player to quit or return to the main menu.

5.1 How to Run Prototype

While the project was exported to web assembly for the purposes of the prototype (requiring a stable internet connection and modern web browser to play), it uses the open source Godot game engine in compatibility and can be exported to any system supported by Godot. The primary way to play the prototype is through a web browser, using the link <https://benhoule.github.io/>. The prototype assumes access to a standard personal computer with a mouse and keyboard.

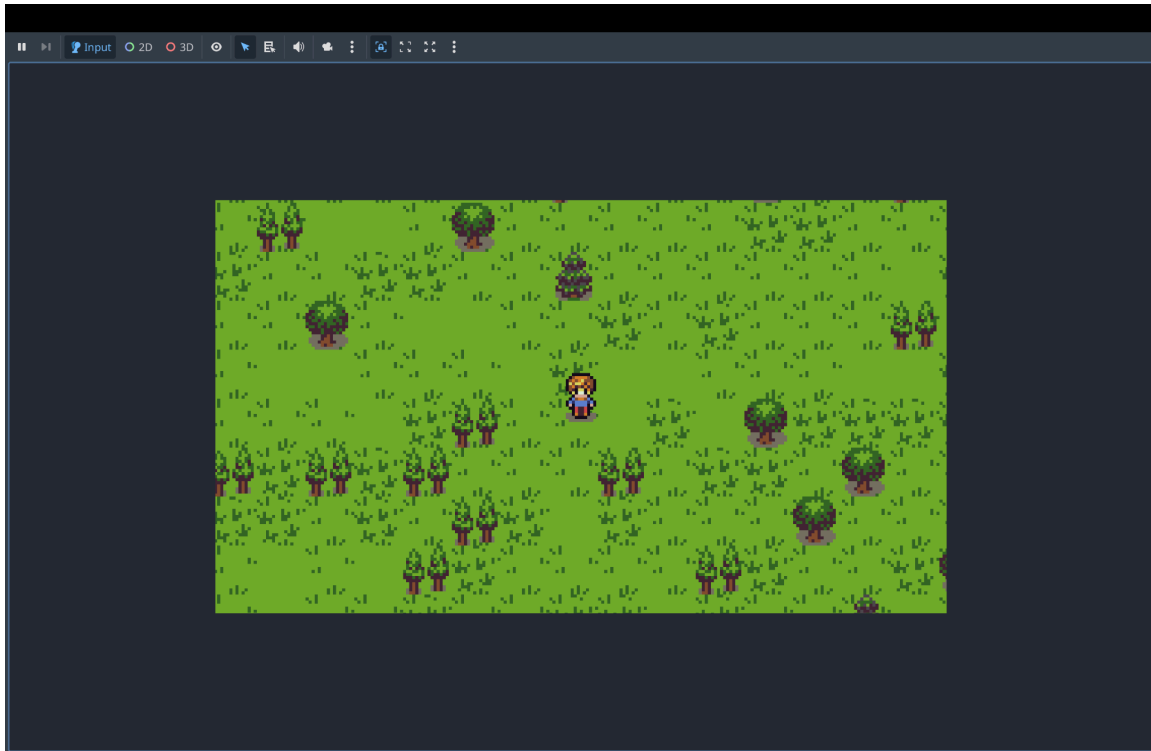
5.2 Sample Scenarios

Level Selection



Upon starting the game, the player will be prompted to select one of three levels, with only the first being unlocked at the start

Island Exploration



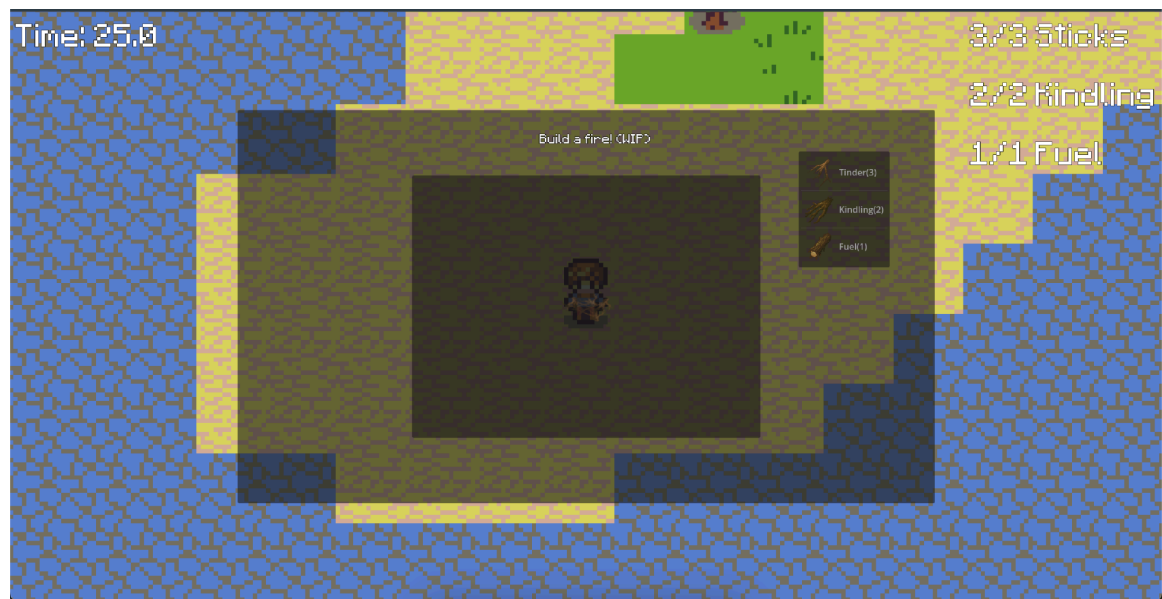
Each level allows the player to move around an island in a top down 2D environment, with various obstacles in the form of trees, water, and shrubs.

Level 1, Collection



The first level has the player gather Sticks, kindling, and fuel from various randomized locations around the island.

Level 1, Firemaking



Upon finding a suitable location to build the fire, the player will be prompted to enter a drag and drop menu to start the fire.

Level 2, Collection



In the second level, players must find five logs suitable for making a shelter.

Level 2, Creating a lean-to



Once five logs are collected, the player must find a clearing suitable for a lean-to, at which point they will be prompted to enter another drop-down menu to create the shelter.

Level 3, Navigation



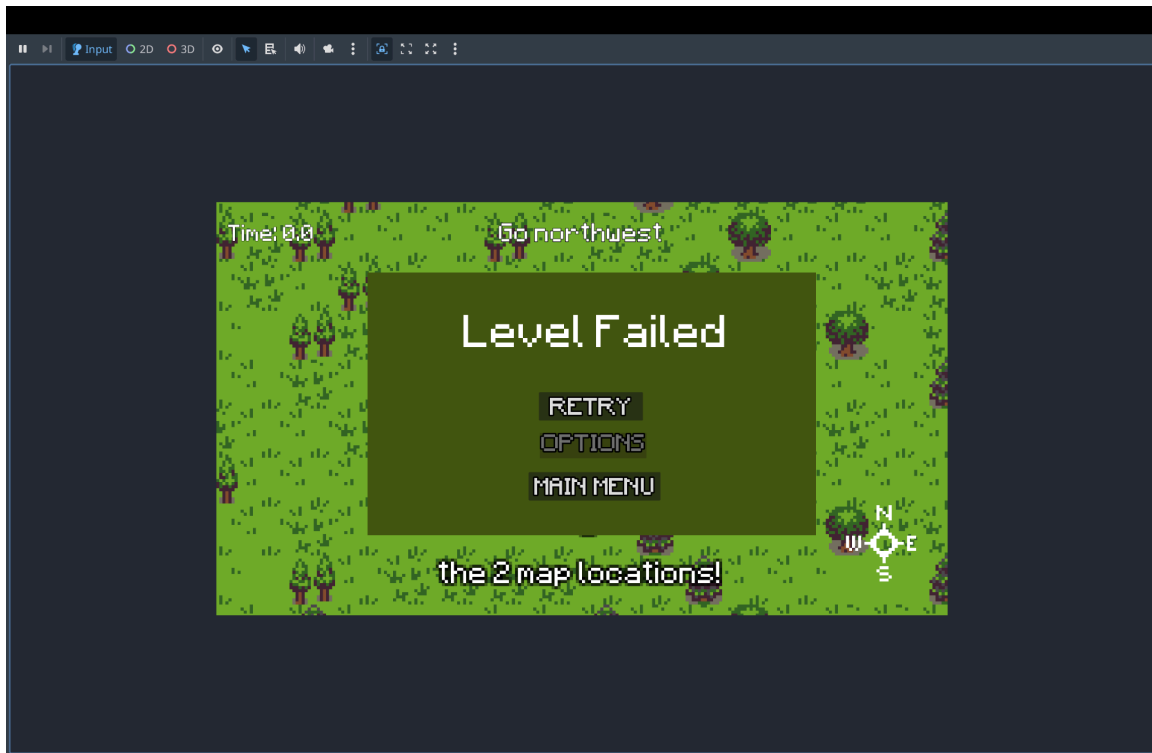
In the third level, the player must use the direction on a compass to find two points on the map.

Objective Completion



Completing a level will allow the user to continue to the next level, or return to the main menu.

Running out of time



Each level will have a time limit which the player must complete the objective under. If they fail to do so, they will be prompted to either start over, or return to the main menu.

6 References

- [1] Endure and Survive Website: <https://camgagne.github.io/swe-website/>
- [2] Photopea <https://www.photopea.com/>
- [3] Draw.io draw.io
- [4] Godot Engine. <https://godotengine.org/>
- [5] Game font: <https://www.dafont.com/bitmap.php>
- [6] Game graphics: <https://opengameart.org/content/tiny-16-basic>

7 Point of Contact

For further information regarding this document and project, please contact **Prof. Daly** at University of Massachusetts Lowell (james_daly at uml.edu). All materials in this document have been sanitized for proprietary data. The students and the instructor gratefully acknowledge the participation of our industrial collaborators.